

CPSC 250 - Programming for Data Manipulation

Test 3 - Solution Set

Topics Covered: Derived classes, constructors and `super()`, inheritance, polymorphism, lists of mixed object types, recursion, binary search, and algorithm efficiency.

Total Points: 80

Part 1 - Multiple Choice (10 points)

1. **b)** To allow one class to reuse and extend another class.
2. **a)** To initialize the base-class portion of the object.
3. **a)** Many classes can respond to the same method call in their own way.
4. **b)** meow
5. **b)** It repeatedly recomputes the same smaller Fibonacci values.

Part 2 - Find the Errors (15 points)

6. Derived class constructor

The `Student` constructor does not call the `Person` constructor, so `self.name` is never created. It should call `super().__init__(name)`.

```
class Person:
    def __init__(self, name):
        self.name = name

class Student(Person):
    def __init__(self, name, student_id):
        super().__init__(name)
        self.student_id = student_id

    def print_info(self):
        print(self.name, self.student_id)

s = Student("Alice", "A123")
s.print_info()
```

7. Method overriding and object creation

There are two errors. First, the overridden `play()` method in `Guitar` is missing `self`. Second, `g = Guitar` stores the class itself, not a new object. It should be `g = Guitar()`.

```
class Instrument:
    def play(self):
        return "sound"

class Guitar(Instrument):
    def play(self):
        return "strum"

g = Guitar()
print(g.play())
```

This prints:

strum

8. Recursive binary search

The function is missing a base case for the situation where the search range is empty. Also, the recursive calls do not remove `mid` from the next search range, so the function can get stuck checking the same middle index forever.

One correct version is:

```
def binary_search(values, target, low, high):
    if low > high:
        return False

    mid = (low + high) // 2

    if values[mid] == target:
        return True

    elif target < values[mid]:
        return binary_search(values, target, low, mid - 1)

    else:
        return binary_search(values, target, mid + 1, high)
```

Part 3 - Code Tracing and Algorithm Reasoning (20 points)

9. Polymorphism tracing

The loop calls `label()` on each object. Python uses the version of `label()` belonging to the actual object type.

```
Item: box
Food: apple (95)
Tool: hammer
```

10. Recursive function tracing

a) `mystery(6)` returns 13.

```
mystery(6) = 6 + mystery(4)
           = 6 + 4 + mystery(2)
           = 6 + 4 + 2 + mystery(0)
           = 6 + 4 + 2 + 1
           = 13
```

b) The base case is `if n <= 1: return 1.`

- c) This behaves more like simple countdown recursion than Fibonacci recursion because each call makes only one smaller recursive call. Fibonacci recursion branches into two recursive calls and repeats much more work.

Part 4 - Code Writing (25 points)

11. Inheritance and super()

One possible solution:

```
class MediaItem:
    def __init__(self, title, year):
        self.title = title
        self.year = year

class Song(MediaItem):
    def __init__(self, title, year, artist):
        super().__init__(title, year)
        self.artist = artist

    def __str__(self):
        return self.title + " by " + self.artist + " (" + str(self.year) + ")"
```

An f-string solution is also fine:

```
def __str__(self):
    return f"{self.title} by {self.artist} ({self.year})"
```

12. Mixed list and polymorphism

One possible solution:

```
items = []

items.append(Plant("hosta", 12.99))
items.append(Plant("fern", 9.50))
items.append(Flower("rose", 18.00, "red"))
items.append(Flower("tulip", 7.25, "yellow"))

for item in items:
    item.print_info()
```

This works because both `Plant` and `Flower` objects have a `print_info()` method. Python chooses the correct version at runtime.

13. Iterative Fibonacci

One possible solution:

```
def fib_iter(n):
    if n == 0:
        return 0
    elif n == 1:
        return 1

    previous = 0
    current = 1

    for i in range(2, n + 1):
        next_value = previous + current
        previous = current
        current = next_value

    return current
```

A shorter solution is also acceptable:

```
def fib_iter(n):
    a = 0
    b = 1

    for i in range(n):
        a, b = b, a + b

    return a
```

Part 5 - Short Answer (10 points)**14. Recursion and end conditions**

Every recursive function needs an end condition, or base case, so that the recursive calls eventually stop. Without a reachable end condition, the function keeps calling itself until Python reaches its recursion limit and raises an error. Conceptually, recursion needs at least one simple case that can be answered directly and one or more recursive cases that move closer to that simple case.

15. Binary search efficiency

Binary search requires the data to be sorted. It works by checking the middle value and then eliminating half of the remaining values each time. This is more efficient than linear search because it does not need to check every item one at a time. A linear search may require about n checks, while binary search requires about $\log_2(n)$ checks.

End of Solution Set